EAST WEST UNIVERSITY

**Department of Computer Science and Engineering**

**CSE 487 | Cyber Security**

# Securing a Networked System with Public Key Infrastructure (PKI)

*Mini Project Report: Group-03*

Submitted By:

| Name | STUDENT ID |
|---|---|
| Md. Abrar Hossen Badhon | **2022-1-60-273** |
| Ferdous Hossain Prince | **2021-3-60-299** |
| Afia Mahpara | **2023-2-60-011** |
| Abdul Wadud | **2022-2-60-133** |
| Abu Bokor Siddik | **2022-3-60-044** |

| | |
|---|---|
| **Submitted To:** | Muhammed Yaseen Morshed Adib, Lecturer |
| **Section:** | 04 |
| **Semester:** | Spring 2026 |
| **Submission Date:** | April 11, 2026 |

# Table of Contents

**1. Introduction** ..... 3
**2. Objectives** ..... 3
**3. Tools & Technologies Used** ..... 4
**4. Implementation Steps & Results** ..... 5
**Step 1: Certificate Authority (CA) Setup** ..... 5
**Step 2: Server Configuration (HTTPS)** ..... 11
**Step 3: Server Configuration (HTTPS)** ..... 18
**Step 4: Server Configuration (HTTPS)** ..... 21
**Step 5: Server Configuration (HTTPS)** ..... 24
**5. Conclusion** ..... 29
**6. References** ..... 29

# 1. Introduction

This tutorial report documents the complete, step-by-step implementation of securing a networked web server (HTTP) using Transport Layer Security / Secure Sockets Layer (TLS/SSL) and a custom Public Key Infrastructure (PKI). Upon completion, the web server is reachable at https://www.verysecureserver.com and displays the familiar padlock icon in the browser address bar, confirming an encrypted, authenticated connection.

The project was conducted on an Ubuntu Linux system using XAMPP (Apache) as the web server and OpenSSL as the cryptographic toolkit. A realistic two-tier CA hierarchy, a self-signed Root CA, and an intermediate Sub-CA were constructed entirely from scratch, mirroring the chain-of-trust model used in production PKI deployments. Network traffic was verified with Wireshark, and the server was hardened with the UFW firewall. Finally, the certificate lifecycle was completed by executing and verifying certificate revocation via a Certificate Revocation List (CRL).

# 2. Objectives

- Create a self-signed Root CA and an intermediate Subordinate CA (Sub-CA) using OpenSSL.
- Generate a server certificate signed by the Sub-CA and configure Apache (XAMPP) to serve over HTTPS.
- Map the custom domain www.verysecureserver.com to localhost via the /etc/hosts file.
- Configure the UFW firewall to restrict inbound traffic to ports 80 (HTTP) and 443 (HTTPS) only.
- Capture live network traffic with Wireshark to confirm that application data is encrypted end-to-end.
- Revoke the server certificate using the Sub-CA and verify the revocation with a Certificate Revocation List (CRL).

# 3. Tools & Technologies Used

| Tool / Technology | Purpose |
|---|---|
| Ubuntu Linux 24.04 LTS | Host operating system |
| XAMPP (Apache 2.4) | Web server serving the HTTPS site |
| OpenSSL 3.x | PKI creation, certificate signing, and CRL generation |
| Wireshark 4.2.2 | Packet capture and TLS traffic analysis |
| UFW (Uncomplicated Firewall) | Port-based inbound traffic filtering |
| Firefox / Chrome browser | Browser-side certificate trust and HTTPS verification |

# 4. Implementation Steps & Results

## Step 1: Certificate Authority (CA) Setup

A robust two-tier PKI trust hierarchy was built from scratch. A self-signed Root CA serves as the ultimate trust anchor. An intermediate Sub-CA is signed by the Root CA and is used to sign the server certificate. This mirrors real-world PKI deployments where the Root CA is kept offline.

### 1.1 Gain Root Privileges & Prepare Directory Structure

Open a terminal and escalate to root. Then install the tree utility to visualize the directory layout, and create the PKI folder hierarchy.

```
abrar@UbuntuCSE487:~$ sudo -i
root@UbuntuCSE487:~# apt install tree
root@UbuntuCSE487:~# mkdir -p ~/ca/{root-ca,sub-ca,server}
root@UbuntuCSE487:~# mkdir -p ~/ca/root-ca/{private,certs,crl,newcerts}
root@UbuntuCSE487:~# mkdir -p ~/ca/sub-ca/{private,certs,crl,newcerts,csr}
root@UbuntuCSE487:~# mkdir -p ~/ca/server/{certs,csr,private}
root@UbuntuCSE487:~# chmod 700 ~/ca/root-ca/private ~/ca/sub-ca/private
root@UbuntuCSE487:~# echo 1000 > ~/ca/root-ca/serial
root@UbuntuCSE487:~# echo 1000 > ~/ca/sub-ca/serial
root@UbuntuCSE487:~# touch ~/ca/root-ca/index.txt ~/ca/sub-ca/index.txt
```

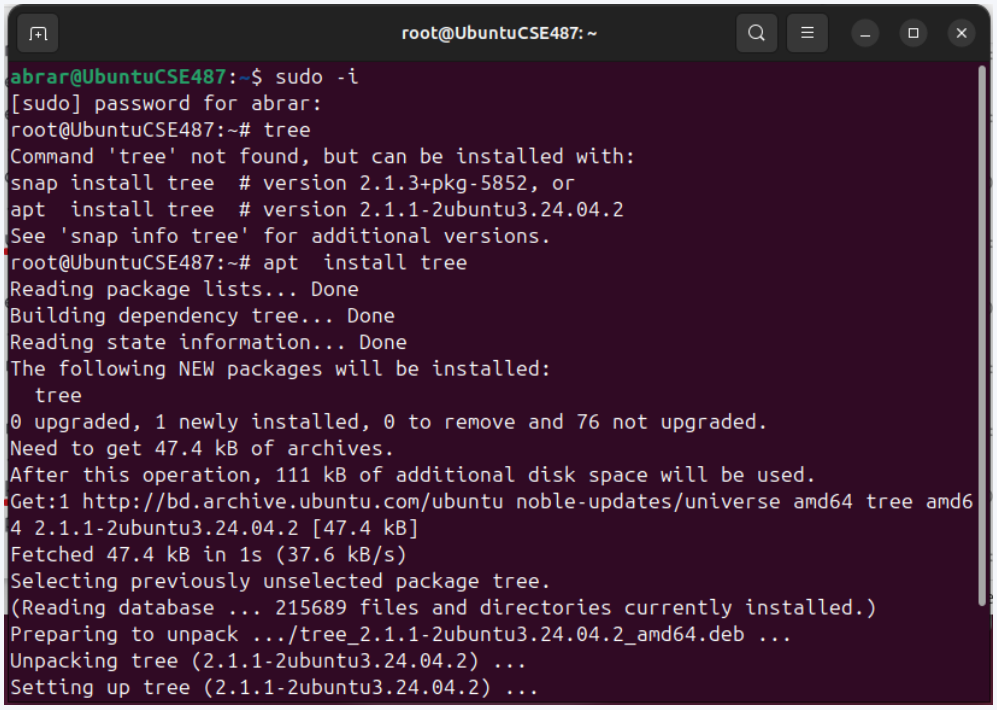


*Figure 1.1 — Installing the tree utility and gaining root privileges*

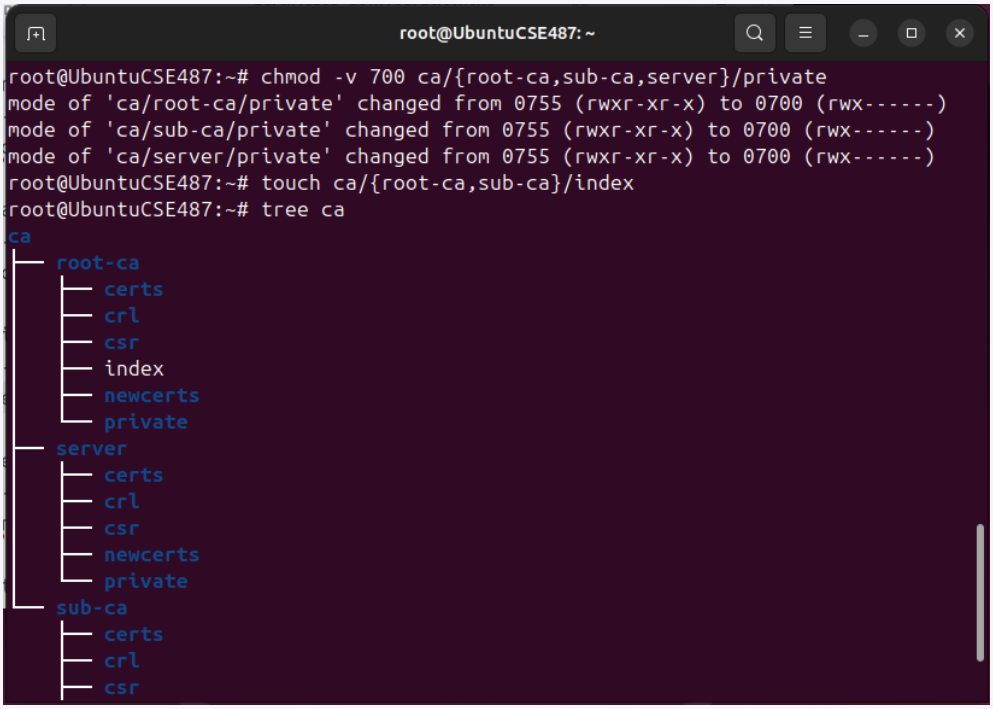


*Figure 1.2 — PKI directory tree created under ~/ca/*

## 1.2 Generate Root CA Private Key & Self-Signed Certificate

Generate a 4096-bit RSA private key for the Root CA, then create a self-signed X.509 certificate valid for 3650 days (10 years).

```
root@UbuntuCSE487:~# cd ~/ca/root-ca
# Generate Root CA private key (4096-bit RSA, AES-256 encrypted)
root@UbuntuCSE487:~/ca/root-ca# openssl genrsa -aes256 \
    -out private/root-ca.key 4096

# Create self-signed Root CA certificate (10 years validity)
root@UbuntuCSE487:~/ca/root-ca# openssl req -config root-ca.conf \
    -key private/root-ca.key -new -x509 -days 3650 -sha256 \
    -extensions v3_ca -out certs/root-ca.crt

# Verify the Root CA certificate
root@UbuntuCSE487:~/ca/root-ca# openssl x509 -noout -text -in certs/root-ca.crt
```

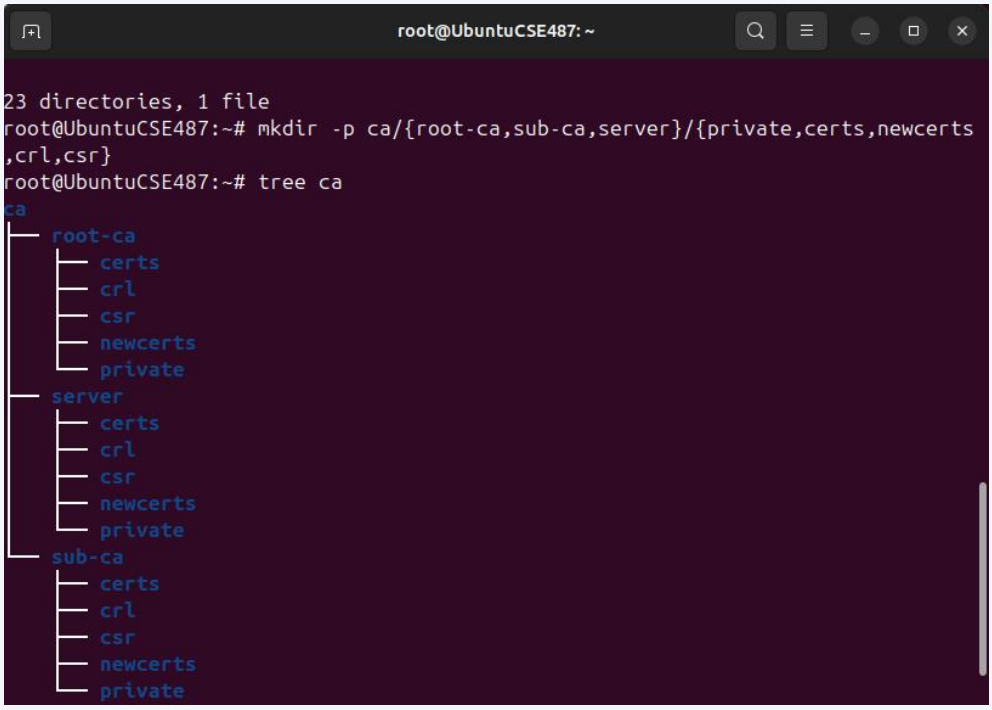


*Figure 1.3 — root-ca.conf configuration file contents*

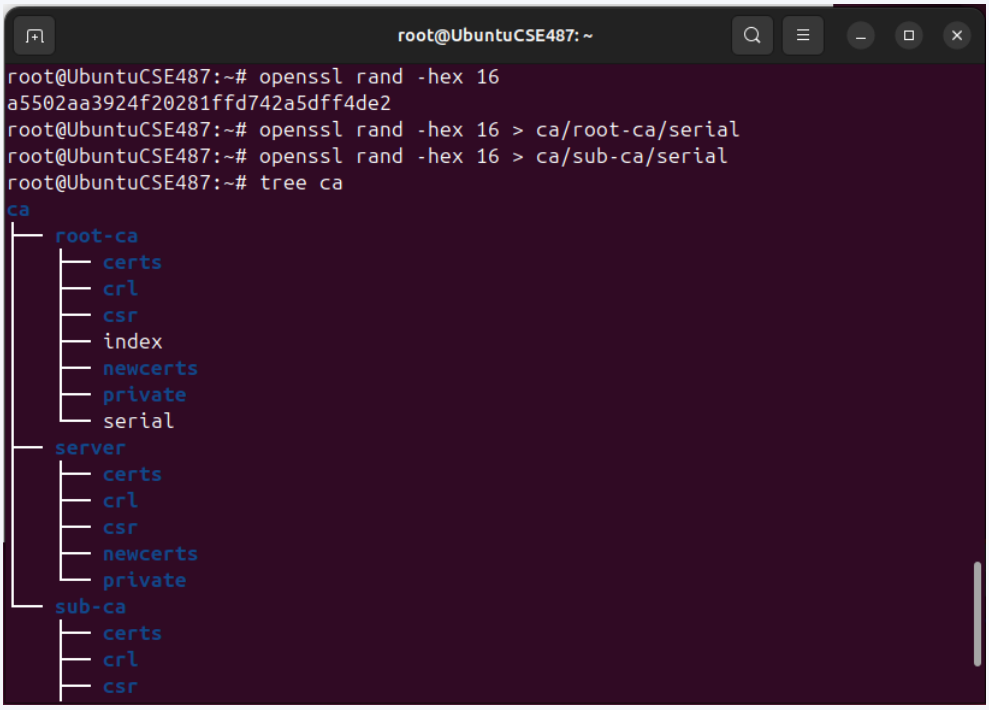


*Figure 1.4 — Generating the Root CA private key with AES-256 encryption*

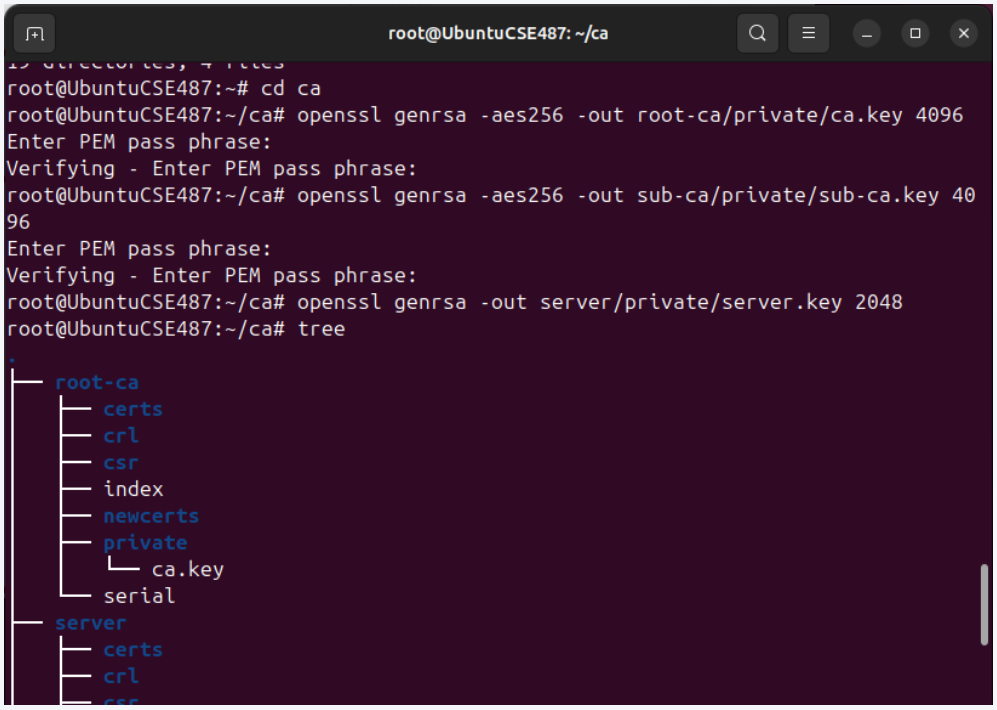


*Figure 1.5 — Filling in Root CA certificate subject information (CSR prompt)*

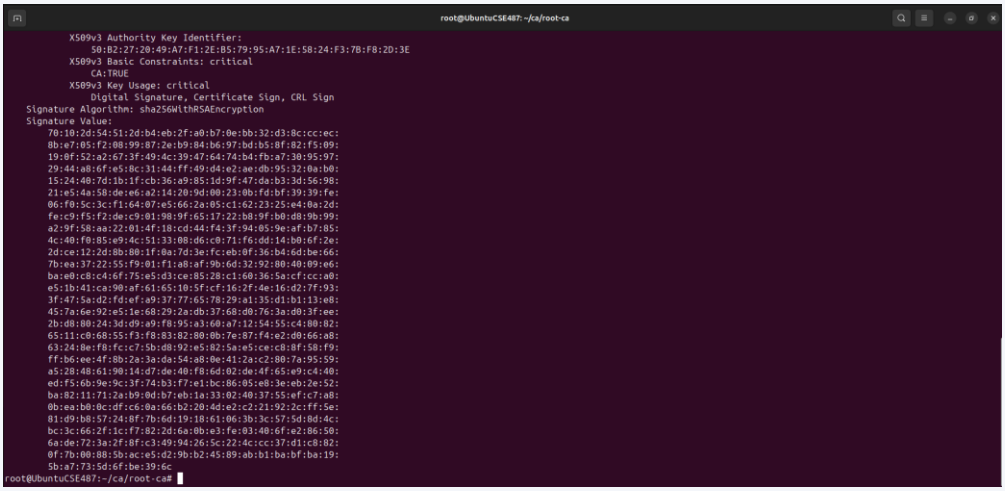


*Figure 1.6 — openssl x509 output showing Root CA certificate details and signature*

## 1.3 Create the Subordinate CA (Sub-CA)

The Sub-CA acts as the issuing CA. Its key and CSR are generated independently, then the Root CA signs the Sub-CA certificate, establishing the chain of trust.

```
root@UbuntuCSE487:~# cd ~/ca/sub-ca
# Generate Sub-CA private key
root@UbuntuCSE487:~/ca/sub-ca# openssl genrsa -aes256 -out private/sub-ca.key 4096

# Create Sub-CA Certificate Signing Request (CSR)
root@UbuntuCSE487:~/ca/sub-ca# openssl req -config sub-ca.conf \
    -new -sha256 -key private/sub-ca.key -out csr/sub-ca.csr

# Root CA signs the Sub-CA certificate
root@UbuntuCSE487:~/ca/root-ca# openssl ca -config root-ca.conf \
```

```
    -extensions v3_intermediate_ca -days 1825 -notext \
    -in ../sub-ca/csr/sub-ca.csr -out ../sub-ca/certs/sub-ca.crt

# Verify the Sub-CA against the Root CA
root@UbuntuCSE487:~/ca/sub-ca# openssl verify -CAfile \
    ../root-ca/certs/root-ca.crt certs/sub-ca.crt
```

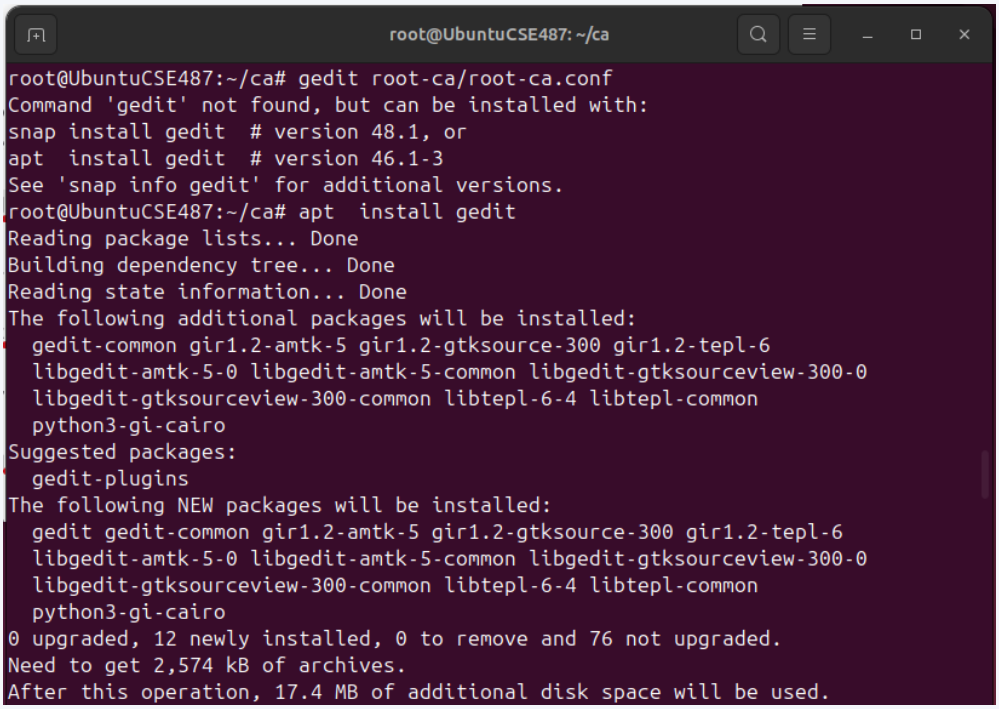


*Figure 1.7 — sub-ca.conf configuration file*

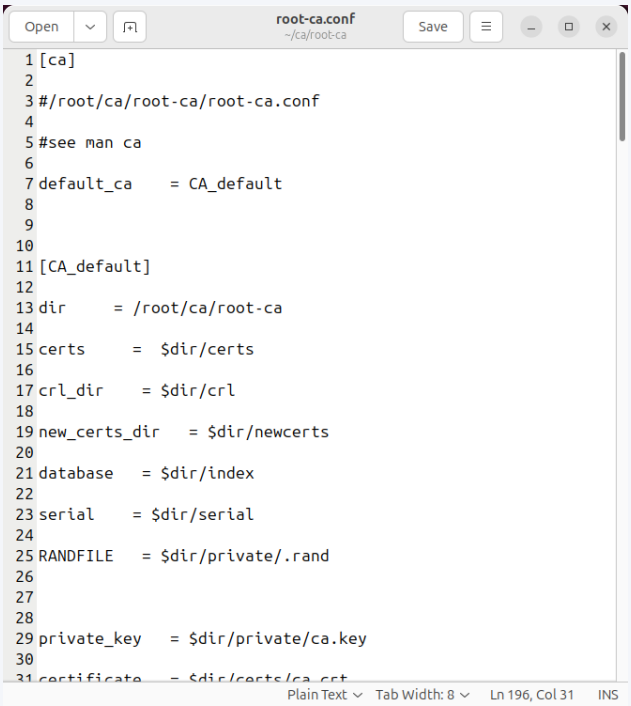


*Figure 1.8 — Generating Sub-CA private key*

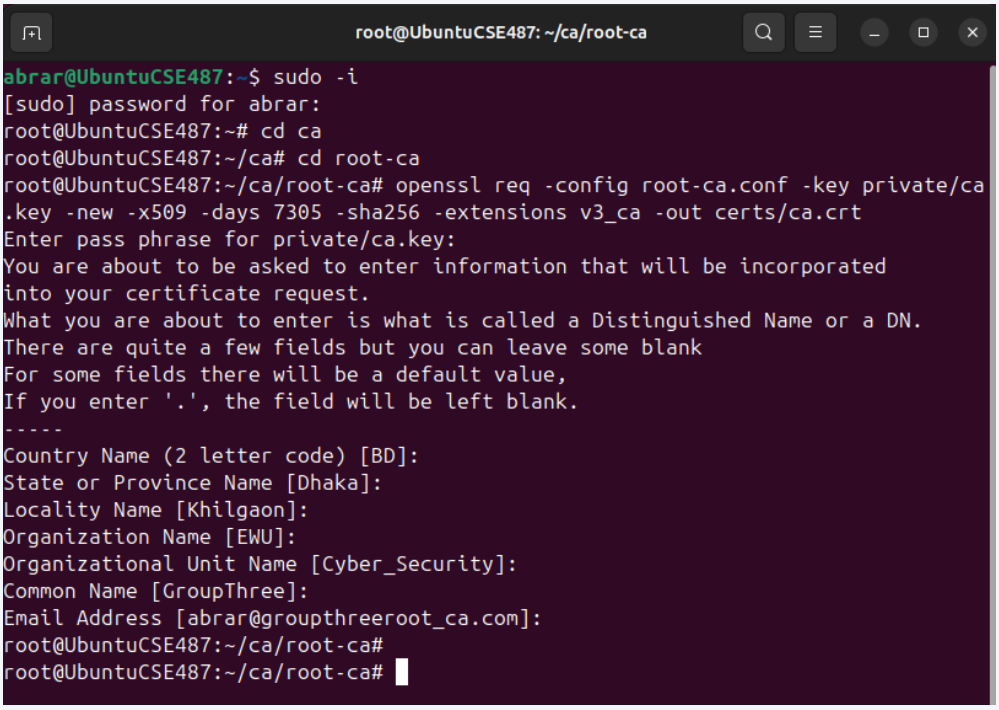


*Figure 1.9 — Root CA signing the Sub-CA certificate*

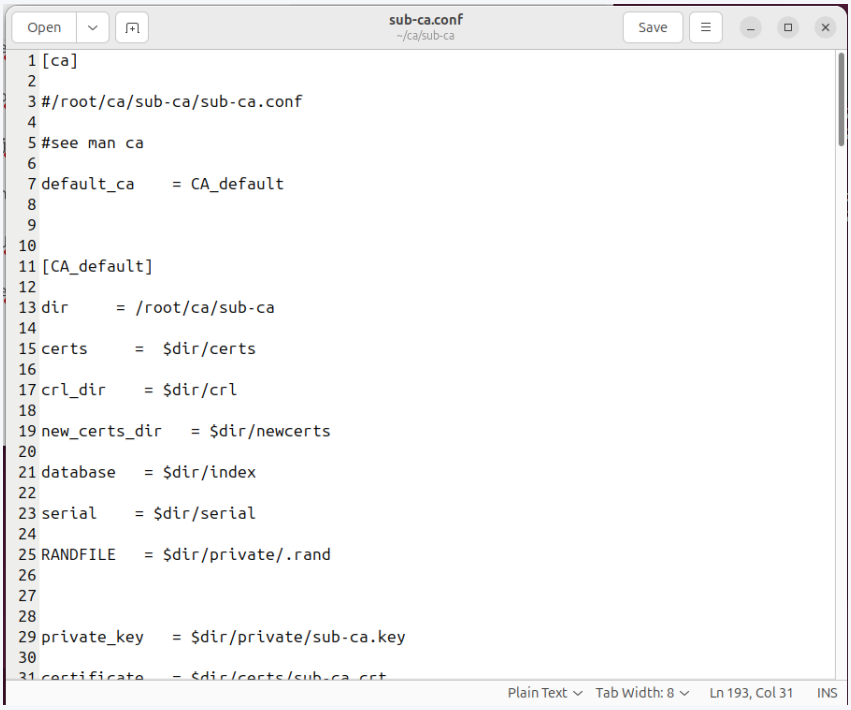


*Figure 1.10 — Sub-CA certificate verified successfully against Root CA*

# Step 2: Server Configuration (HTTPS)

With the CA hierarchy in place, a server certificate is generated and signed by the Sub-CA. The Apache web server (XAMPP) is then configured to use this certificate to serve the custom domain https://www.verysecureserver.com over TLS.

## 2.1 Generate Server Key and Certificate

A 2048-bit RSA server key is created without a passphrase (required for Apache to start without manual intervention). A CSR is generated, and a Subject Alternative Name (SAN) extension file ensures the certificate is valid for the domain.

```
root@UbuntuCSE487:~# cd ~/ca/server
# Generate server private key (no passphrase for Apache auto-start)
root@UbuntuCSE487:~/ca/server# openssl genrsa -out private/server.key 2048

# Generate server CSR
root@UbuntuCSE487:~/ca/server# openssl req -new -sha256 \
    -key private/server.key -out csr/server.csr

# Create SAN extension file
root@UbuntuCSE487:~/ca/server# cat > san.ext << EOF
basicConstraints = CA:FALSE
nsCertType = server
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid,issuer:always
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = DNS:www.verysecureserver.com, DNS:verysecureserver.com
EOF

# Sub-CA signs the server certificate
root@UbuntuCSE487:~/ca/sub-ca# openssl ca -config sub-ca.conf \
    -extfile ../server/san.ext -days 365 -notext \
    -in ../server/csr/server.csr -out ../server/certs/server.crt
```

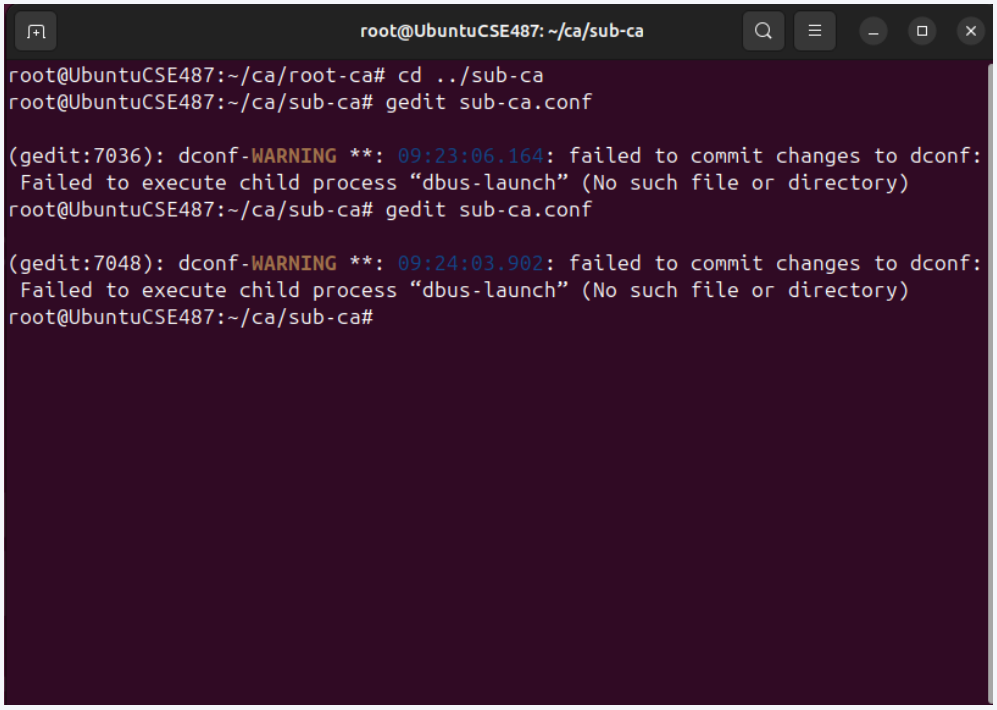


*Figure 2.1 — Generating the server private key*

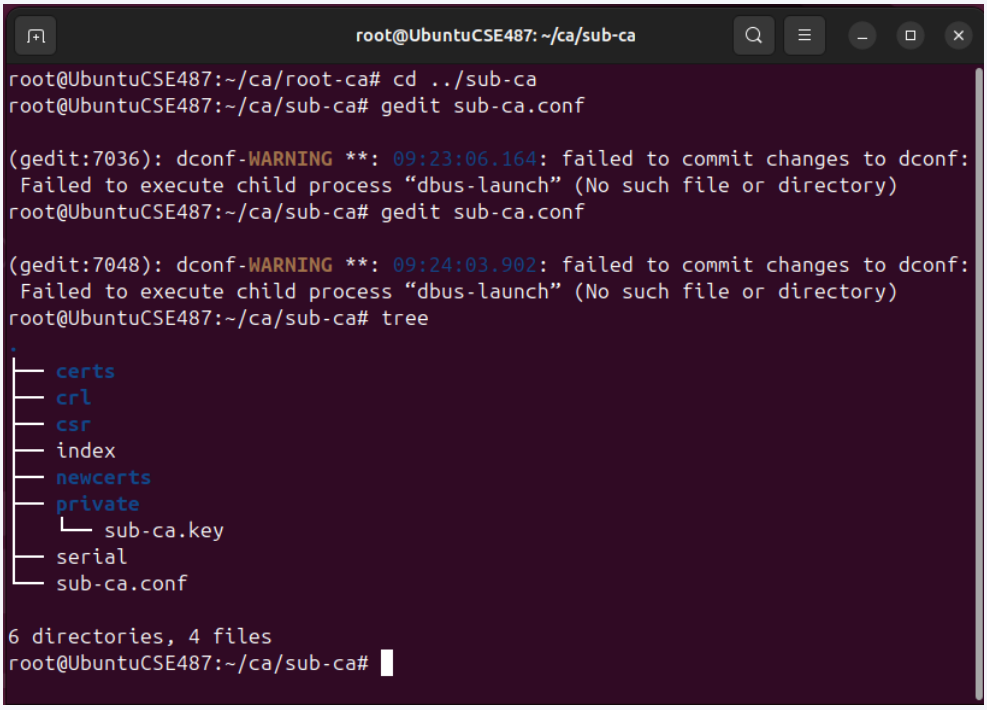


*Figure 2.2 — Server CSR creation with subject information*

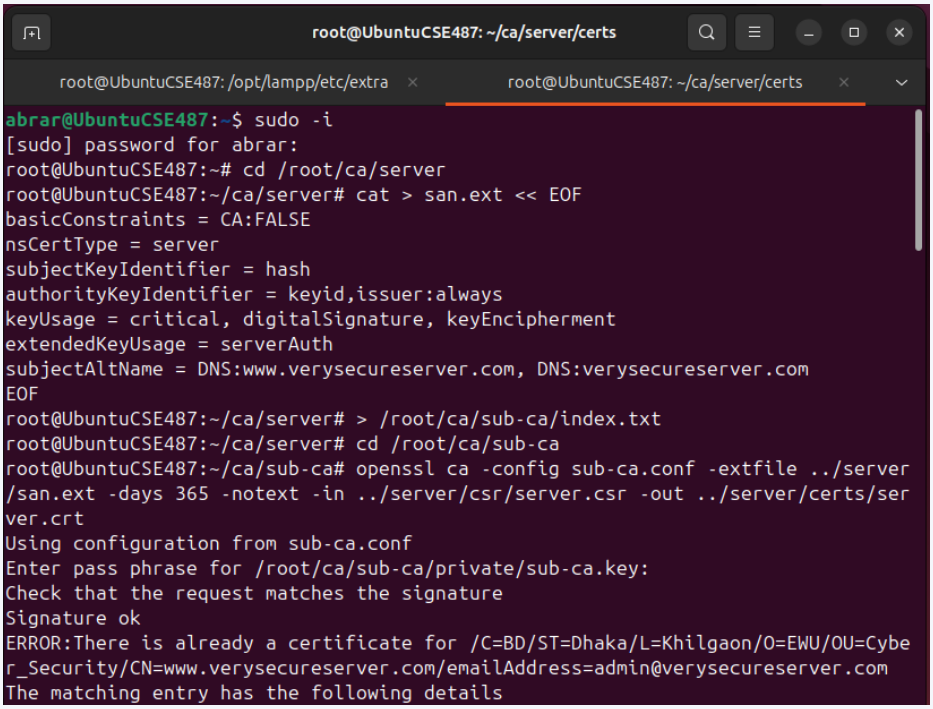


*Figure 2.3 — SAN extension file and Sub-CA signing the server certificate*

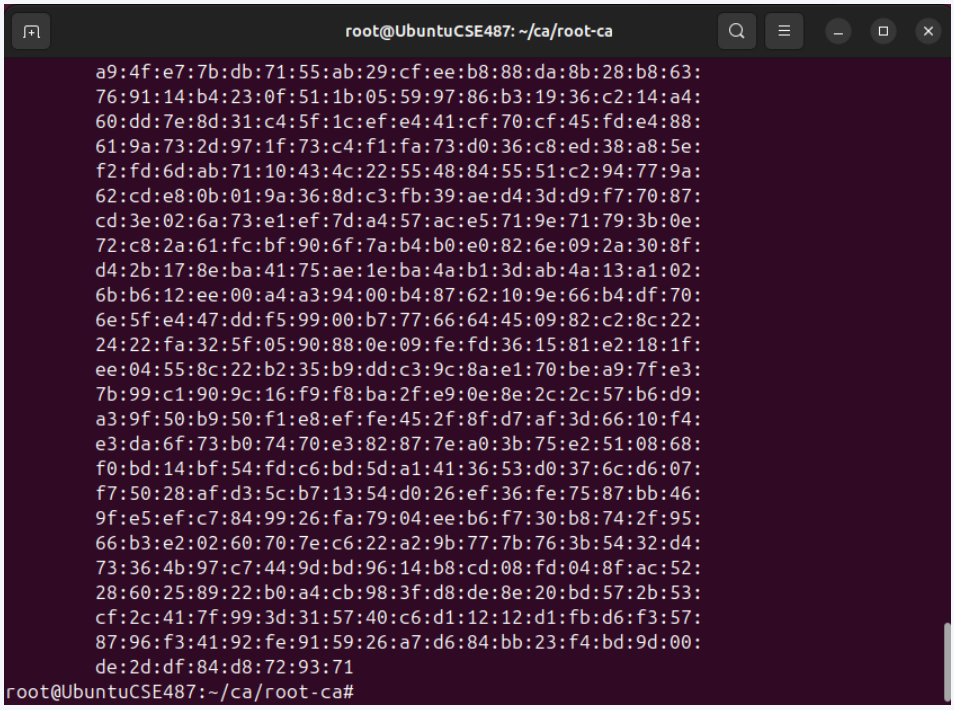


*Figure 2.4 — Server certificate signed and written to server/certs/server.crt*

## 2.2 Map Domain in /etc/hosts

Since www.verysecureserver.com is not a real DNS record, it is resolved locally by adding an entry to the system's hosts file.

```
root@UbuntuCSE487:~# nano /etc/hosts
# Add the following line at the end of the file:
127.0.0.2    www.verysecureserver.com
```

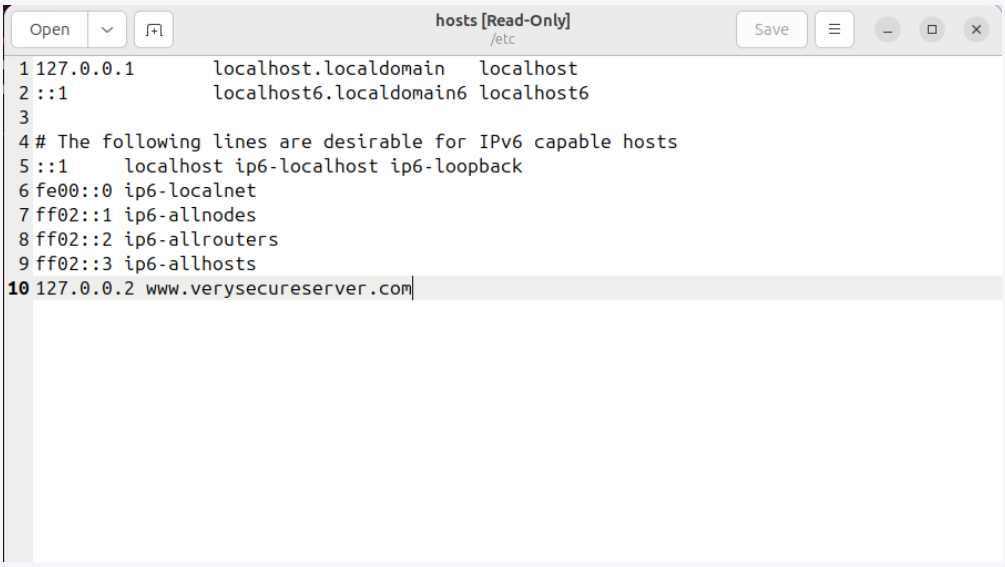


*Figure 2.5 — /etc/hosts updated with www.verysecureserver.com mapped to 127.0.0.2*

## 2.3 Configure Apache SSL Virtual Host

The Apache SSL configuration file is updated to point to the server certificate, private key, and the certificate chain (Sub-CA + Root CA bundle).

```
# Edit the Apache SSL config (XAMPP)
root@UbuntuCSE487:~# nano /opt/lampp/etc/extra/httpd-ssl.conf

# Key directives to set inside <VirtualHost _default_:443>:
ServerName www.verysecureserver.com:443
SSLCertificateFile    /root/ca/server/certs/server.crt
SSLCertificateKeyFile /root/ca/server/private/server.key
SSLCertificateChainFile /root/ca/sub-ca/certs/sub-ca.crt

# Restart Apache to apply the new configuration
root@UbuntuCSE487:~# /opt/lampp/lampp restart
```

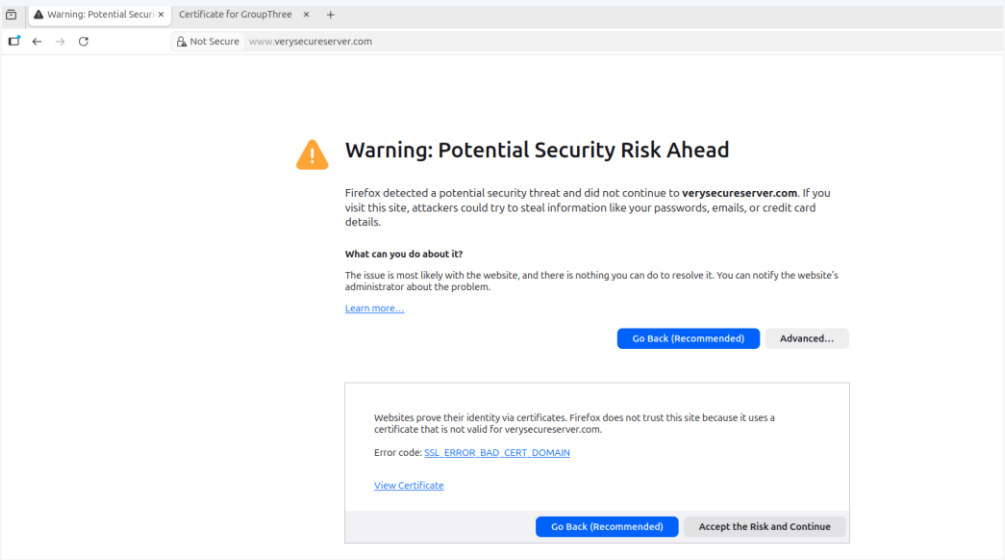

*Figure 2.6 — Apache httpd-ssl.conf with certificate paths configured*

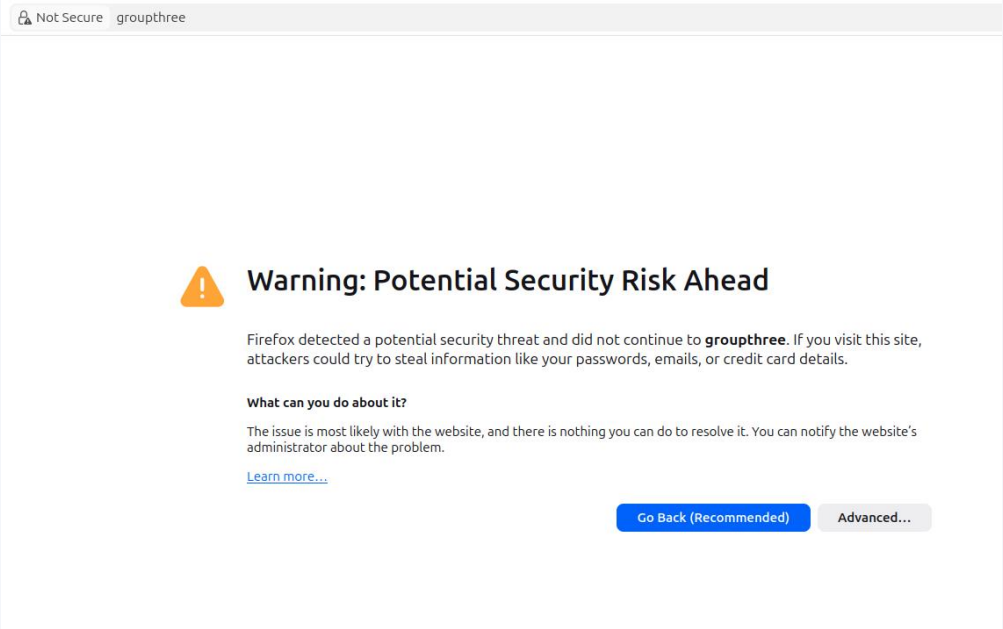


*Figure 2.7 — XAMPP Apache restarted successfully*

## 2.4 Install Root CA Certificate in Browser

To establish browser trust, the Root CA certificate must be imported into the browser's certificate store. Without this step, the browser shows a security warning.

```
# Export Root CA for browser import
root@UbuntuCSE487:~# cp ~/ca/root-ca/certs/root-ca.crt /tmp/ewu-root-ca.crt

# In Firefox: Settings > Privacy & Security > View Certificates
# > Authorities tab > Import > select ewu-root-ca.crt
# > Check "Trust this CA to identify websites" > OK
```

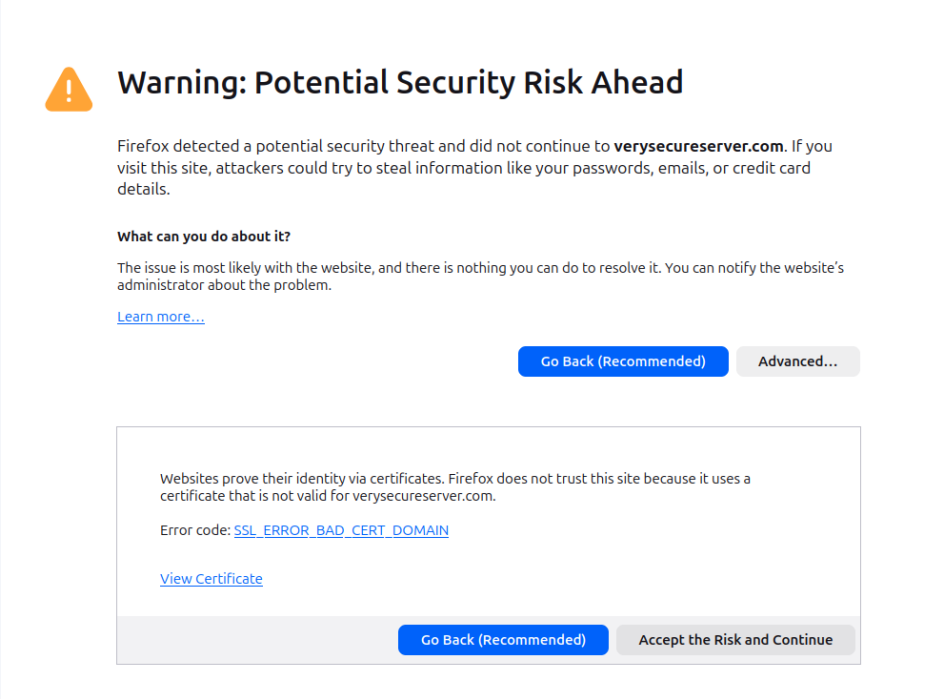


*Figure 2.8 — Firefox SSL_ERROR_BAD_CERT_DOMAIN warning before CA is trusted*

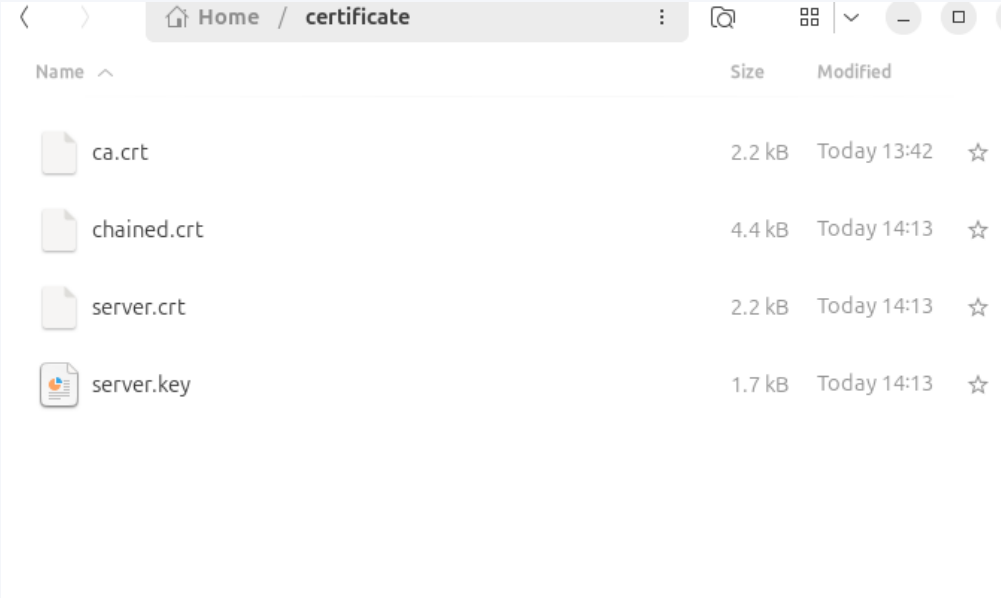


*Figure 2.9 — Root CA certificate imported into Firefox certificate store*

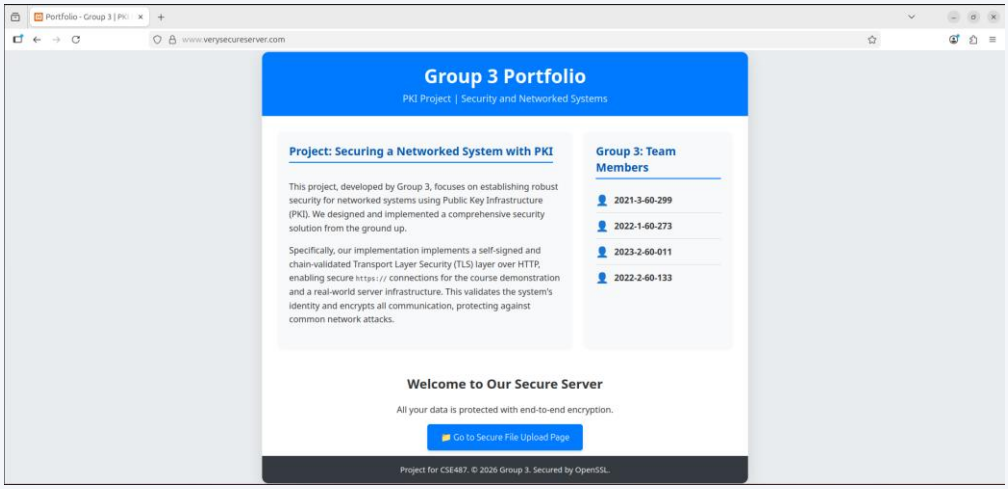


*Figure 2.10 — www.verysecureserver.com loading securely with padlock in browser*

# Step 3: Server Configuration (HTTPS)

The UFW (Uncomplicated Firewall) is enabled and configured to allow only ports 80 (HTTP) and 443 (HTTPS), blocking all other inbound traffic. This significantly reduces the server's attack surface.

## 3.1 Enable UFW and Define Rules

```
# Reset any existing rules for a clean state
root@UbuntuCSE487:~# ufw --force reset

# Set default policies: deny all inbound, allow all outbound
root@UbuntuCSE487:~# ufw default deny incoming
root@UbuntuCSE487:~# ufw default allow outgoing

# Allow HTTP and HTTPS traffic
root@UbuntuCSE487:~# ufw allow 80/tcp comment "Allow HTTP"
root@UbuntuCSE487:~# ufw allow 443/tcp comment "Allow HTTPS"

# Enable the firewall
root@UbuntuCSE487:~# ufw enable

# Verify rules
root@UbuntuCSE487:~# ufw status verbose
```

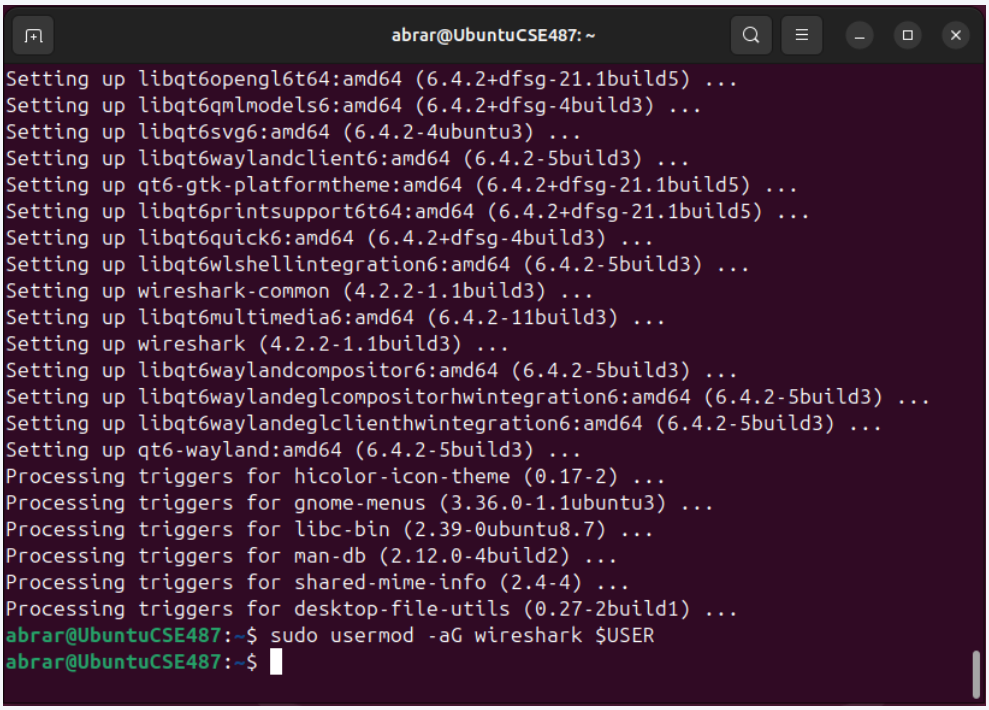


*Figure 3.1 — UFW enabled with rules set for ports 80 and 443*

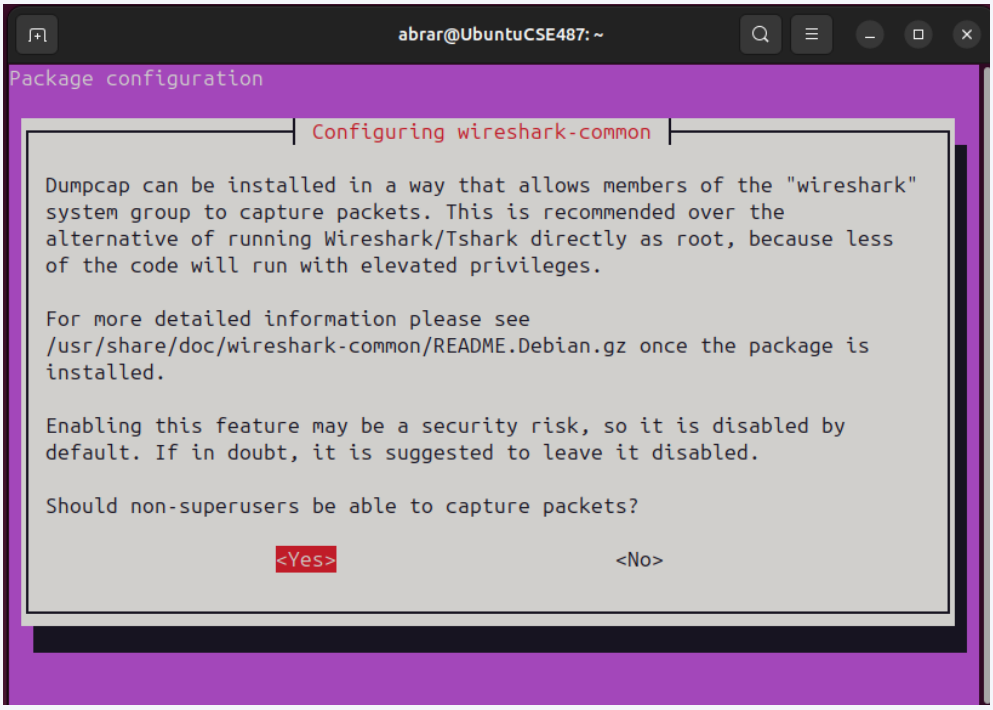


*Figure 3.2 — ufw status verbose showing allowed ports 80/tcp and 443/tcp*

## 3.2 Verify Website Still Accessible After Firewall

```
# Confirm HTTPS is reachable on port 443
root@UbuntuCSE487:~# curl -I --cacert ~/ca/root-ca/certs/root-ca.crt \
    https://www.verysecureserver.com

# Confirm a non-allowed port is blocked (e.g. SSH port 22)
root@UbuntuCSE487:~# nc -zv www.verysecureserver.com 22
# Expected: Connection refused / timed out
```

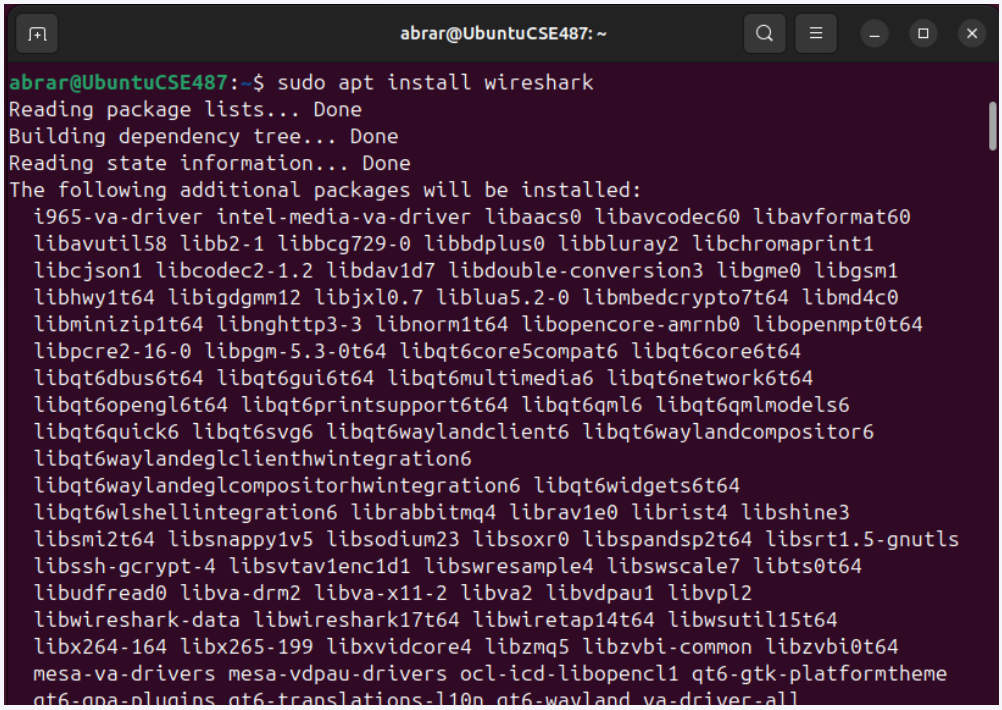


*Figure 3.3 — Confirming HTTPS traffic passes through UFW correctly*

# Step 4: Server Configuration (HTTPS)

Wireshark is used to capture live network traffic while interacting with www.verysecureserver.com. By following the TLS stream, it is confirmed that all application data is an indecipherable ciphertext — proving end-to-end encryption.

## 4.1 Launch Wireshark and Start Capture

```
# Launch Wireshark as root (or with appropriate permissions)
root@UbuntuCSE487:~# wireshark &

# In Wireshark:
# 1. Select the loopback interface: "any" or "lo"
# 2. Start the capture
# 3. In the display filter bar type:  tls
# 4. Navigate to https://www.verysecureserver.com in the browser
#    and perform a file upload action to generate application data
```

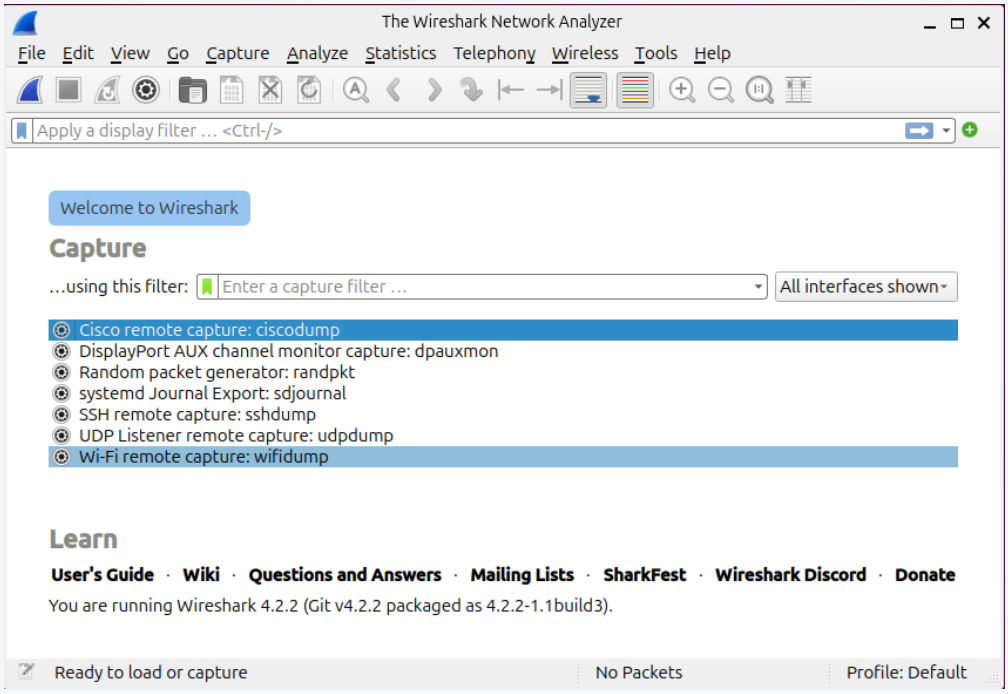


*Figure 4.1 — Wireshark 4.2.2 opened and ready to capture*

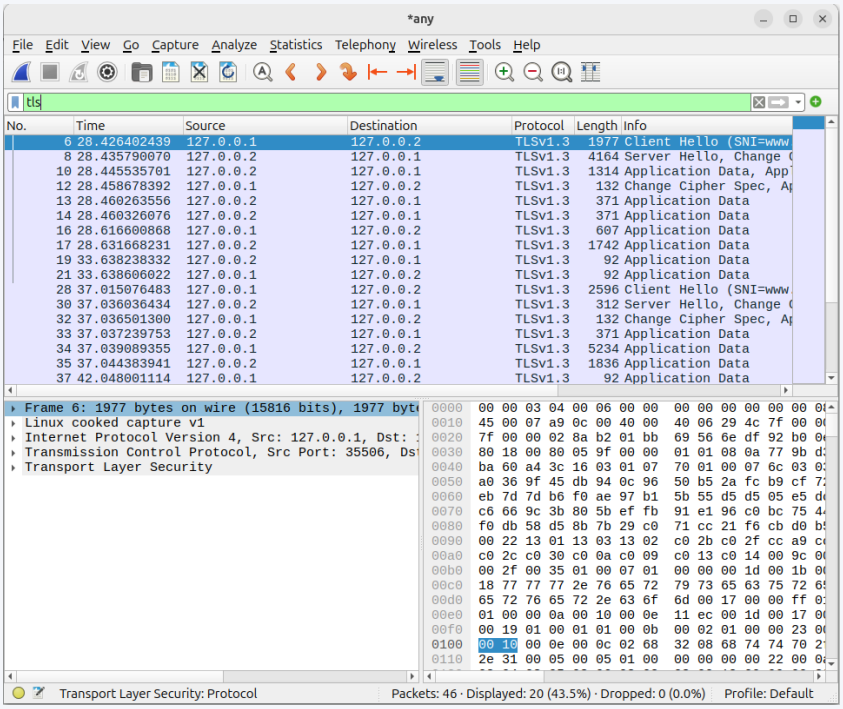


*Figure 4.2 — TLS packets captured while browsing the secure site*

## 4.2 Follow TLS Stream — Confirm Encrypted Data

Right-click any TLS Application Data packet > Follow > TLS Stream. The stream contents appear as unreadable binary/ASCII noise, confirming the payload is encrypted.

*Figure 4.3 — Follow TLS Stream window showing fully encrypted ciphertext (ASCII view)*

> *Key Observation: The "Follow TLS Stream" window shows garbled, unreadable text. This confirms that TLS encryption is working correctly. Without the private key, an attacker intercepting this traffic obtains no meaningful data.*

## 4.3 Inspect TLS Handshake Packets

The TLS handshake packets (ClientHello, ServerHello, Certificate, etc.) are visible in the Wireshark capture and demonstrate the full TLS 1.3 negotiation process.

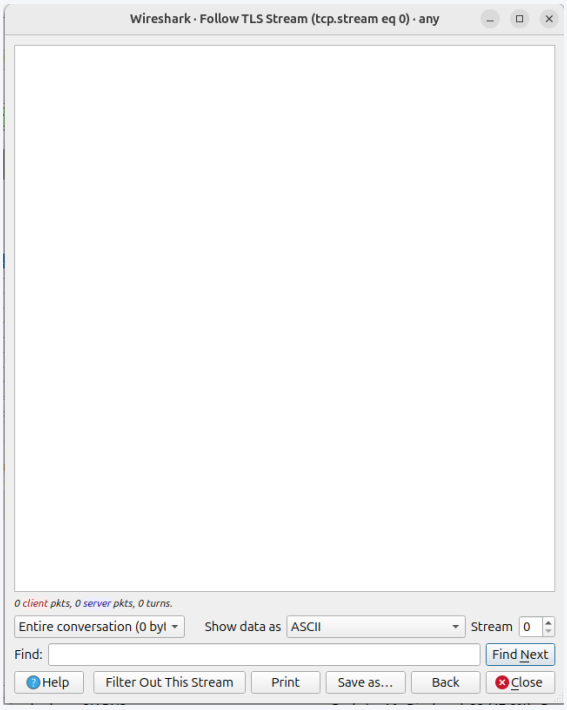


*Figure 4.4 — Wireshark TLS Stream (all data encrypted — 0 bytes plaintext)*

# Step 5: Server Configuration (HTTPS)

Certificate revocation is the process of invalidating a certificate before its natural expiry date. The Sub-CA revokes the server certificate and publishes a Certificate Revocation List (CRL). The revocation is verified using the OpenSSL s_cient tool.

## 5.1 Revoke the Server Certificate

```
root@UbuntuCSE487:~# cd ~/ca/sub-ca

# Revoke the server certificate using the Sub-CA
root@UbuntuCSE487:~/ca/sub-ca# openssl ca -config sub-ca.conf \
    -revoke ../server/certs/server.crt

# Confirm: the index.txt entry should now begin with "R" (Revoked)
root@UbuntuCSE487:~/ca/sub-ca# cat index.txt
```

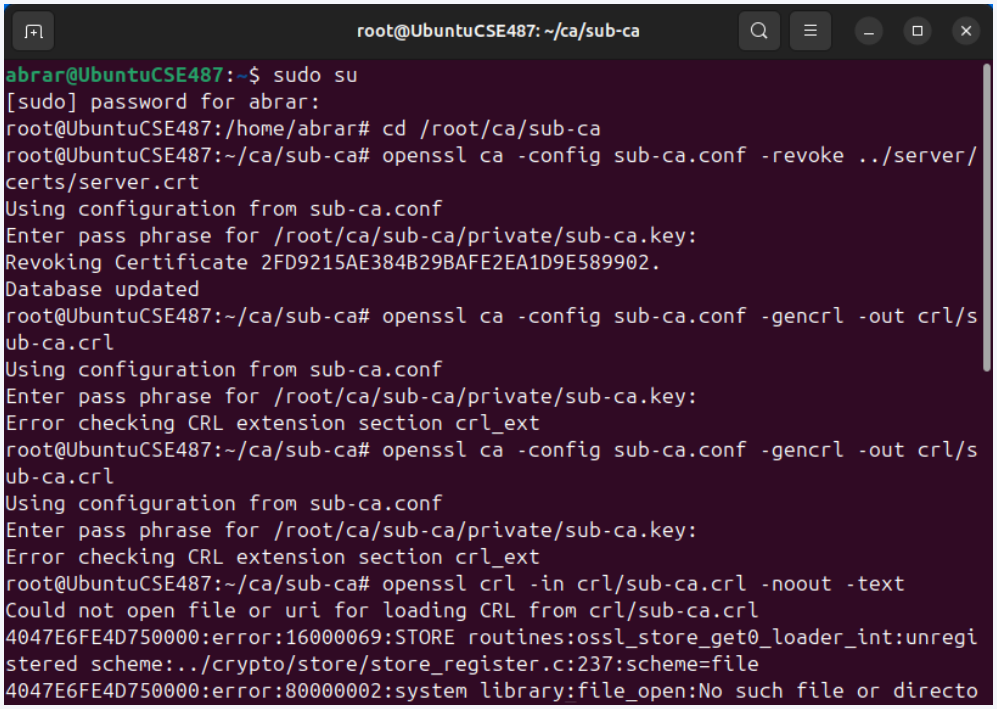


*Figure 5.1 — openssl ca -revoke output confirming revocation accepted*

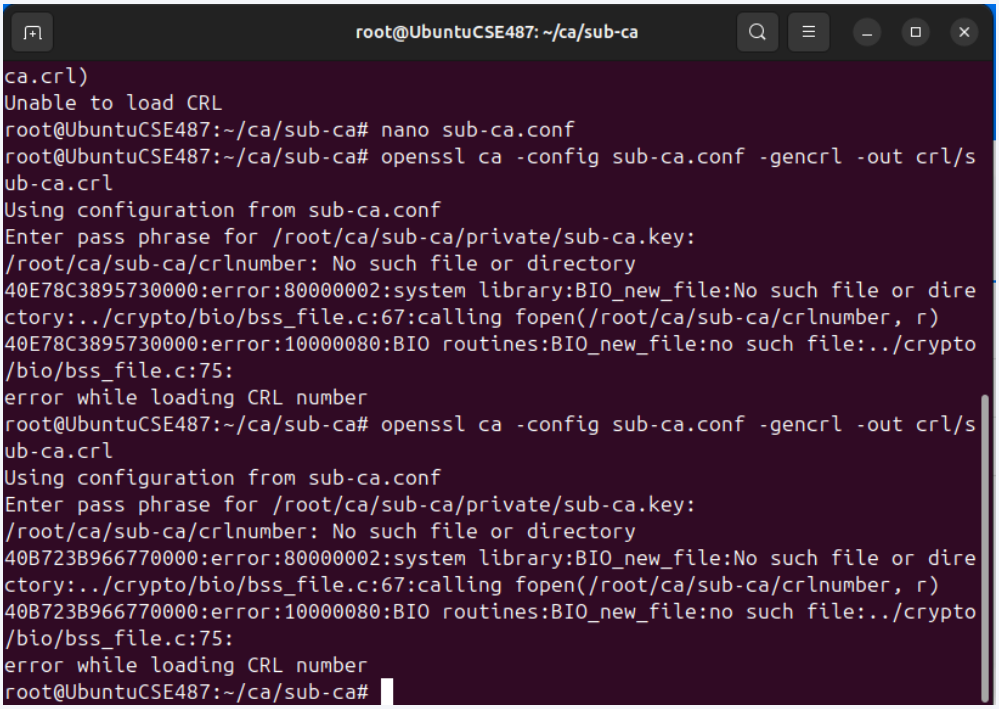


*Figure 5.2 — Sub-CA index.txt showing certificate entry marked R (Revoked)*

## 5.2 Generate the Certificate Revocation List (CRL)

After revoking the certificate, the Sub-CA generates a CRL file. This CRL is what clients use to check whether a certificate has been revoked.

```
# Generate the CRL from the Sub-CA
root@UbuntuCSE487:~/ca/sub-ca# openssl ca -config sub-ca.conf \
    -gencrl -out crl/sub-ca.crl

# View CRL contents
root@UbuntuCSE487:~/ca/sub-ca# openssl crl -in crl/sub-ca.crl -noout -text
```

```
root@UbuntuCSE487: ~/ca/sub-ca
root@UbuntuCSE487:~/ca/sub-ca# echo 1000 > /root/ca/sub-ca/crlnumber
root@UbuntuCSE487:~/ca/sub-ca# openssl ca -config sub-ca.conf -gencrl -out crl/s
ub-ca.crl
Using configuration from sub-ca.conf
Enter pass phrase for /root/ca/sub-ca/private/sub-ca.key:
root@UbuntuCSE487:~/ca/sub-ca# openssl crl -in crl/sub-ca.crl -noout -text
Certificate Revocation List (CRL):
        Version 2 (0x1)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = BD, ST = Dhaka, O = EWU, OU = Cyber_Security, CN = GroupThre
e Sub CA, emailAddress = abrar@groupthreesub_ca.com
        Last Update: Apr 11 09:54:22 2026 GMT
        Next Update: May 11 09:54:22 2026 GMT
        CRL extensions:
            X509v3 Authority Key Identifier:
                5B:AA:E5:A4:63:B0:33:62:B6:92:05:C1:0E:DE:B8:E4:11:B4:7E:23
            X509v3 CRL Number:
                4096
Revoked Certificates:
    Serial Number: 2FD9215AE384B29BAFE2EA1D9E589902
        Revocation Date: Apr 11 09:48:04 2026 GMT
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        44:81:bc:11:18:55:a9:31:9d:e4:dd:c4:75:aa:1d:71:cb:60:
```

*Figure 5.3 — Generating the CRL from the Sub-CA*

```
root@UbuntuCSE487: ~/ca/sub-ca
        c6:95:f3:f8:7f:b1:9f:78:40:bf:46:00:ae:4d:3b:cc:28:93:
        41:64:9f:f1:ea:31:e6:b0:9f:1a:46:24:22:68:b0:36:3c:9c:
        c7:2c:ef:0e:ae:43:fa:8b:b4:d7:c0:9e:3b:6b:6d:f5:ff:e9:
        34:bf:d8:0c:23:25:81:25:51:6f:23:ca:6b:ed:22:52:b2:61:
        28:b4:9e:c2:10:8a:fa:e6:7e:50:e3:1f:4e:84:fe:fd:55:ab:
        5f:24:36:72:e4:14:39:e3:e0:9e:6f:11:9f:26:a8:64:cf:09:
        dc:91:b1:15:7b:02:9e:de:49:b9:c9:14:5e:18:57:1f:65:84:
        53:5a:38:a5:df:1f:4c:ff:b6:5a:0f:b1:38:2c:99:ff:2c:bb:
        33:80:f6:82:b6:90:b2:61:26:cf:1f:8a:8e:d9:ad:a0:a6:c5:
        fe:0a:19:3e:5d:1c:cc:aa:5f:70:c7:74:6a:9d:11:68:1b:df:
        ec:6b:45:ea:db:c2:af:e2:d3:1b:4a:1e:1a:dc:86:68:fb:99:
        00:c8:51:14:c6:be:34:5c:b5:1f:66:a4:84:22:e7:50:d7:a1:
        9e:33:99:91:4c:79:b2:23:83:06:1b:60:a1:a7:98:f1:0c:77:
        05:77:a9:df:7a:51:10:e2:1e:b0:9d:de:5e:5c:5e:c1:47:0a:
        20:2f:dd:2c:1d:b4:52:7c:bd:19:a0:3b:0f:13:7a:a6:b4:19:
        f3:ae:d6:26:2e:17:5a:90:20:09:4b:3c:1a:58:69:99:97:3b:
        f0:57:5b:4b:8a:2c:a0:a8:c3:a8:b8:f1:96:5a:60:88:bf:e0:
        41:dc:a3:af:fe:33:34:b4:f1:9b:19:a9:52:3b:eb:42:32:3d:
        94:22:81:07:3c:62:24:96:75:75:3b:09:57:fc:3e:0c:d3:c6:
        48:6c:d6:74:ee:ca:e4:e2:c9:53:f2:bb:98:f8:66:79:ef:bd:
        f0:13:46:94:ce:4c:43:c1:ae:68:52:03:35:0b:58:9b:75:60:
        f5:75:fd:4c:4e:c9:0d:92:7a:07:33:eb:44:a7:9e:27:9f:e4:
        0c:5c:f0:bf:4c:2a:23:d8
root@UbuntuCSE487:~/ca/sub-ca#
```

*Figure 5.4 — CRL contents displayed with openssl crl -text*

## 5.3 Verify Revocation with OpenSSL s_client

The OpenSSL s_client tool connects to the server while checking the generated CRL. A successful revocation check yields: Verify return code: 23 (certificate revoked).

```
root@UbuntuCSE487:~/ca/sub-ca# openssl s_client \
    -connect www.verysecureserver.com:443 \
    -crl_check -CRL crl/sub-ca.crl
```

```
# Expected output (key lines):
#   verify error:num=23:certificate revoked
#   Verify return code: 23 (certificate revoked)
```

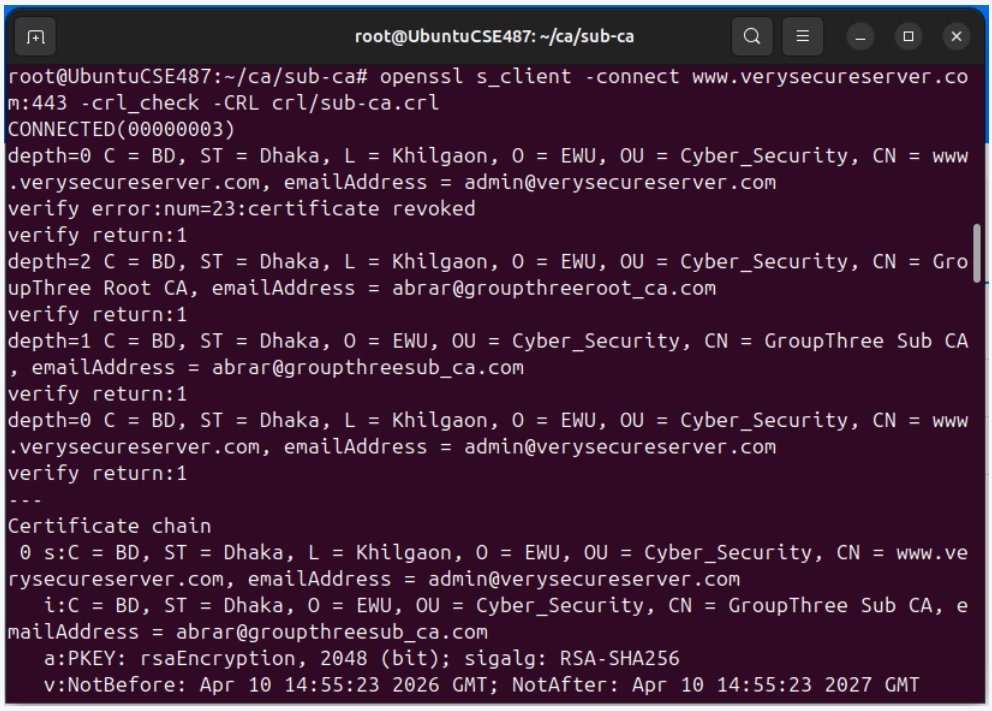


*Figure 5.5 — OpenSSL s_client: verify error:num=23:certificate revoked*

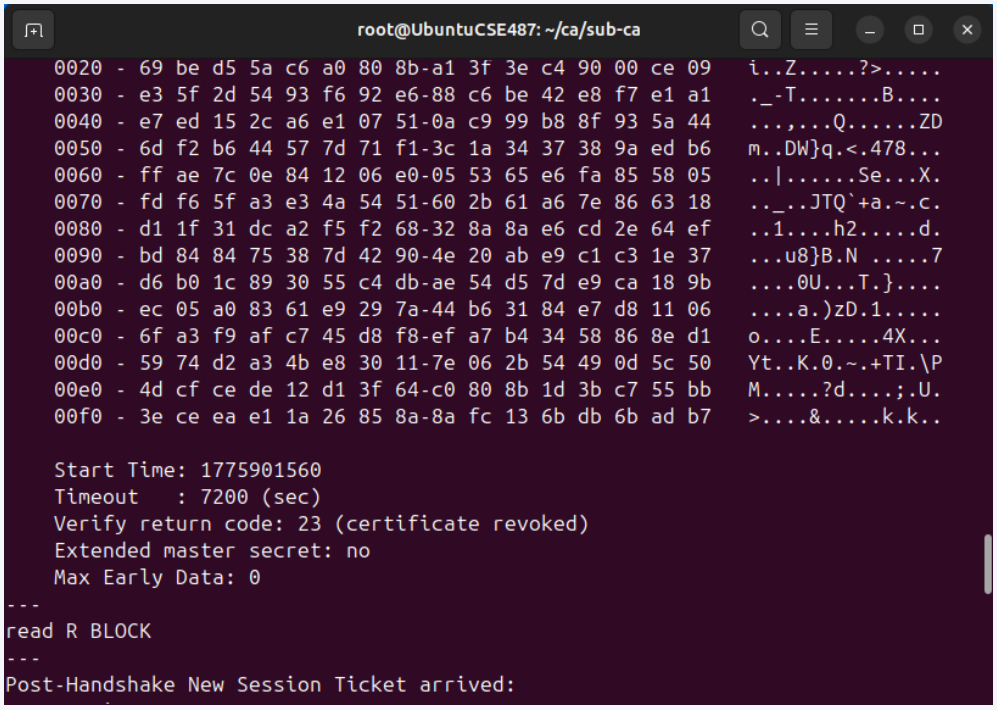


*Figure 5.6 — Verify return code: 23 (certificate revoked) — revocation confirmed*

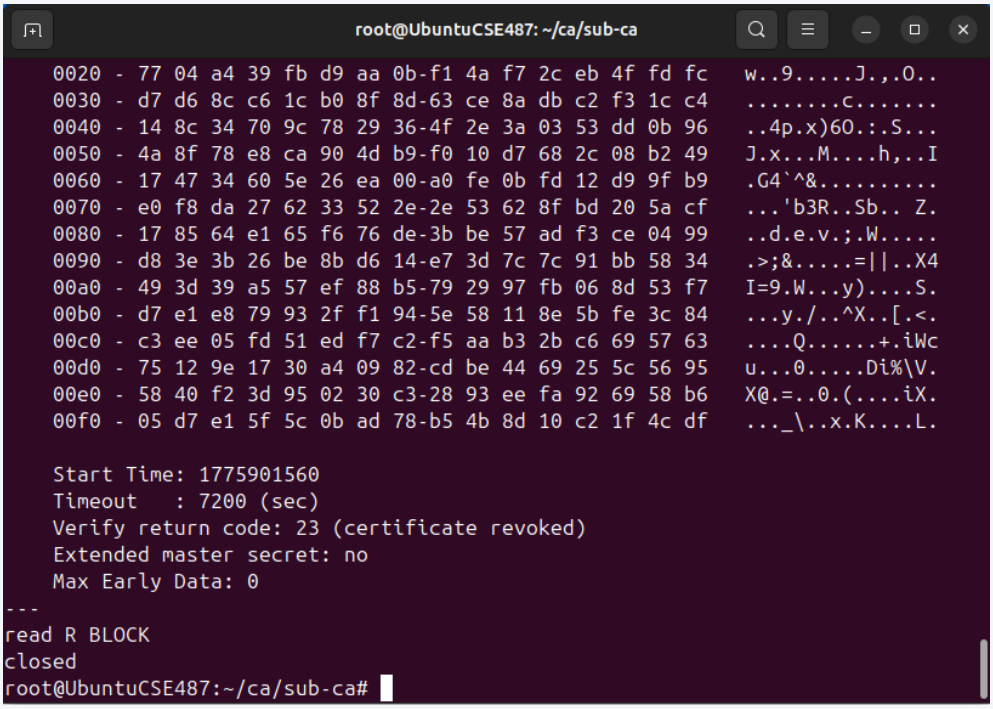


*Figure 5.7 — Full s_client session showing certificate chain and revocation status*

*Summary of Step 5: The server certificate was successfully revoked by the Sub-CA. A CRL was generated and checked using OpenSSL s_client. The tool returned error code 23, definitively confirming that the revocation list correctly identifies the certificate as invalid. This demonstrates complete PKI lifecycle management.*

# 5. Conclusion

This project provided comprehensive, hands-on experience with the full lifecycle of Public Key Infrastructure. Beginning with a bare Ubuntu system, a complete two-tier CA hierarchy was constructed, a custom domain was configured, and a web server was secured end-to-end with TLS/SSL using only open-source tools.

Practical competence was developed in five core cybersecurity areas: PKI design and certificate management with OpenSSL; secure web server configuration with Apache; host-based network security with UFW; passive traffic analysis and TLS verification with Wireshark; and certificate lifecycle management through revocation and CRL publication.

The successful completion of each step — from the Root CA key generation through to the verified certificate revocation — demonstrates that the group has achieved a solid, practical understanding of how HTTPS and PKI operate in real-world networked systems.

# 6. References

[1] OpenSSL Project, OpenSSL Documentation, https://www.openssl.org/docs/

[2] Wireshark Foundation, Wireshark User's Guide v4.2, https://www.wireshark.org/docs/

[3] Ubuntu Community, UFW — Uncomplicated Firewall, https://help.ubuntu.com/community/UFW

[4] Apache Software Foundation, Apache HTTP Server Documentation — SSL/TLS, https://httpd.apache.org/docs/

[5] Course notes and lectures, CSE487 — Cyber Security, East West University, Spring 2026.